



Prac 6 Cell Phone Inner Join Search Adding Values 2025 Complete App

Applied Systems Implementation 1 (University of KwaZulu-Natal)

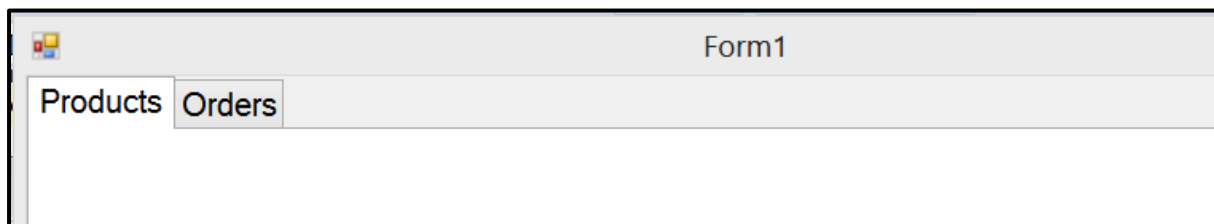


Scan to open on Studocu

Prac 6 – Inner Join, Search and Generate Order

PART ONE (COMPULSORY)

1. Create a user interface (UI) that consists of a form with a **Tab control** that is docked (choose the fill option) with 2 tabs as shown below:



2. Create 2 tables that consists of Cell **Products** and Cell **Suppliers**. The table names could be TblProduct and TblSupplier; The design and data for both these tables are illustrated below :

Column Name	Data Type
ProdID	int
Description	varchar(50)
Price	decimal(6, 2)
Provider	varchar(50)

Deterministic	Yes
DTS-published	No
Full-text Specification	No
Has Non-SQL Server Subscriber	No
Identity Specification	Yes
(Is Identity)	Yes
Identity Increment	1
Identity Seed	1

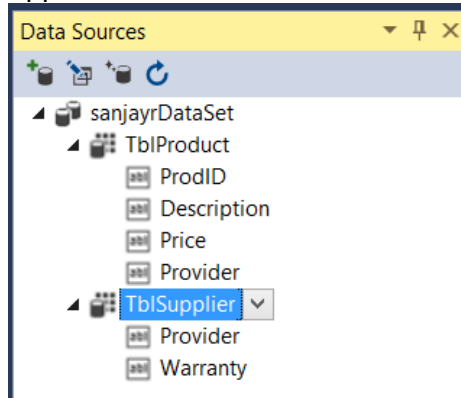
ProdID	Description	Price	Provider
1	Samsung Galaxy S8	7500.00	MTN
2	Samsung Galaxy S9	14000.00	Cell C
3	Samsung Note 9	18500.00	Vodacom
4	Apple iPhone 8	7800.00	MTN
5	Apple iPhone 9	9500.00	MTN
6	Apple iPhone 10	15000.00	Vodacom
7	Apple iPhone 10	14500.00	Cell C
10	Huawei Mate 10	12000.00	Cell C
11	Huawei Mate 10	13000.00	Vodacom

Column Name	Data Type
Provider	varchar(50)
Warranty	varchar(50)

Provider	Warranty
MTN	24 months
Vodacom	36 months
Cell C	18 months

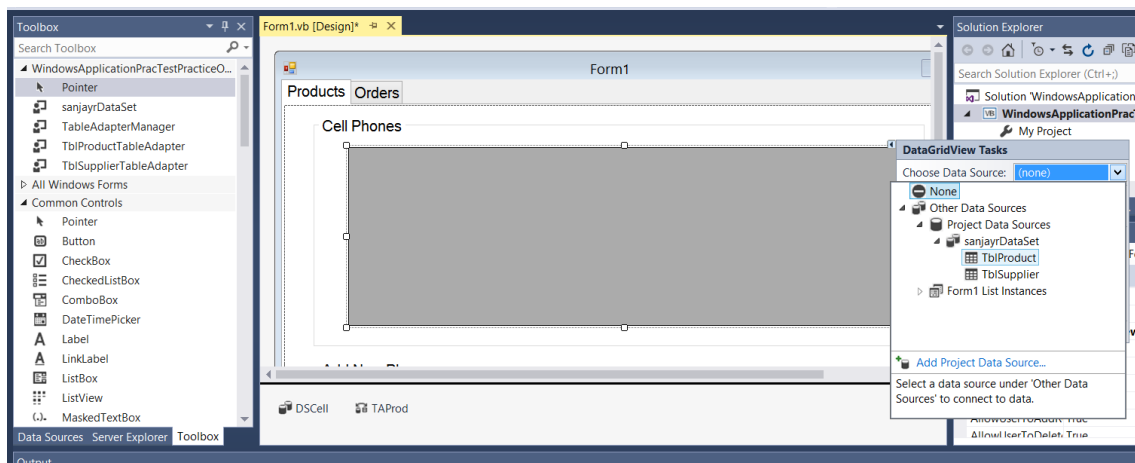
Note: If the prices are expected to be large...then a better option will be to set the size of the Amount/Price field to decimal(10,2)

3. In the VS application, create a new data source and establish a link to both the tables



Rebuild the Application so that you are able to view the data connection objects in the Toolbox.

Add a Group Box labelled CellPhones and a Gridview. Attach the Gridview to the Table Adapter for the Product table



Load the Cell Phone data into the Dataset

```
private void Form5_Load(object sender, EventArgs e)
{
    taCellPhone.Fill(dsCell.TblProduct);
}
```

Display all the Cell Phones in stock via a GridView as shown below.

Form1

Products Orders

Cell Phones

	ProdID	Description	Price	Provider
▶	1	Samsung Galaxy S8	7500.00	MTN
	2	Samsung Galaxy S9	14000.00	Cell C
	3	Samsung Note 9	18500.00	Vodacom
	4	Apple iPhone 8	7800.00	MTN
	5	Apple iPhone 9	9500.00	MTN
	6	Apple iPhone 10	15000.00	Vodacom
	7	Apple iPhone 10	14500.00	Cell C
	10	Huawei Mate 10	12000.00	Cell C
	11	Huawei Mate 10	13000.00	Vodacom
*				

- Extend the interface to allow for the adding of a new cell phone onto the stock. The Provider for the cell phone is selected from a Combobox. As soon as the new phone is added to the stock, it should be immediately displayed in the Gridview. The Insertion sequence is shown below

WindowsApplicationPracTestPracticeOrderSupplier (Running) - Microsoft Visual Studio

Form1

Products Orders

Cell Phones

	ProdID	Description	Price	Provider
▶	1	Samsung Galaxy S8	7500.00	MTN
	2	Samsung Galaxy S9	14000.00	Cell C
	3	Samsung Note 9	18500.00	Vodacom
	4	Apple iPhone 8	7800.00	MTN
	5	Apple iPhone 9	9500.00	MTN
	6	Apple iPhone 10	15000.00	Vodacom
	7	Apple iPhone 10	14500.00	Cell C
	10	Huawei Mate 10	12000.00	Cell C
	11	Huawei Mate 10	13000.00	Vodacom
*				

Add New Phone

Provide Phone Description
Huawei Mate 20 Pro

Enter Phone Price
22000

Select Provider
Vodacom
MTN
Vodacom
Cell C
Add Phone to Stock

```

1 Public Class Form1
2     Private Sub Form1_Load(sender As Object, e As EventArgs) Handles MyBase.Load
3         TAProd.Fill(DSCell.TblProduct)
4     End Sub
5
6     Private Sub btnAddNewPhone_Click(sender As Object, e As EventArgs) Handles btnAddNewPhone.Click
7         TAProd.Insert(txtPhoneDescription.Text, txtPhonePrice.Text)
8         TAProd.Fill(DSCell.TblProduct)
9     End Sub
10 End Class
11

```

In order to achieve this effect, 2 things are required:

- Attach the Combo Box to the Provider table

ComboBox Tasks

Use Data Bound Items

Data Binding Mode

Data Source: tblSupplierBin

tblProductBindingSource1

tblSupplierBindingSource1

tblSupplierBindingSource2

Other Data Sources

Project Data Sources

sanjayDataSet

TblProduct

TblSupplier

Add Project Data Source...

Selecting a project data source creates an instance on the form and binds to...

Properties

comboBoxProvider System.Windows.Forms.ComboBox

RightToLeft: No

Size: 235, 37

Sorted: False

TabIndex: 3

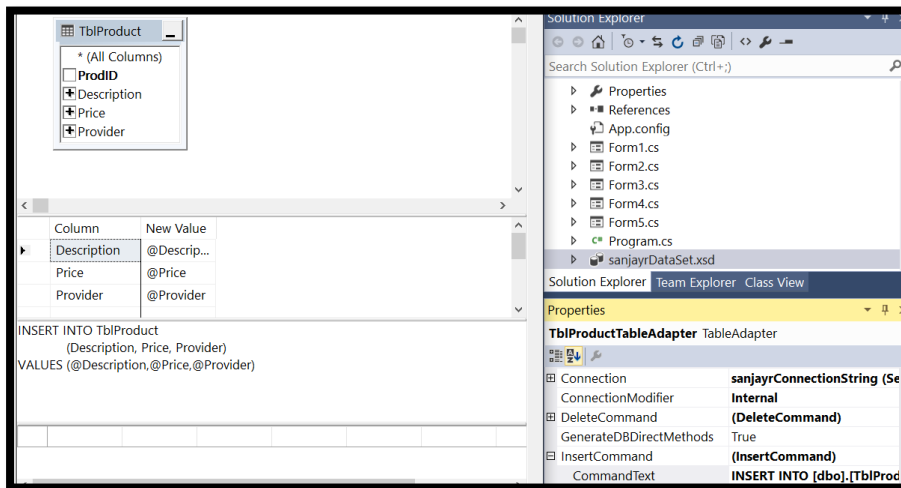
TabStop: True

Tag

Text

UseWaitCursor: False

- Invoke the pre-written Insert method that is available via the Product tableAdapter;



*taCellProd.Insert(txtCellDescrip.Text, Convert.ToDecimal(txtCellPrice.Text),
 comboCellProvider.SelectedValue.ToString());*

```
private void btnAddNewCellPhone_Click(object sender, EventArgs e)
{
    try
    {
        taCellProd.Insert (txtDescription.Text, Convert.ToDecimal(txtPrice.Text),
        comboProvider.SelectedValue.ToString());
        MessageBox.Show("New Phone has been Added");
        taCellProd.Fill(dsCell.TblProduct);
    }
    catch (Exception)
    {
        MessageBox.Show("New Phone has NOT been Added");
    }
}
```

This will work fine....however, you may want to ***prompt the user to confirm the insertion of the new record***...in which case...use the DialogResult class

```
private void btnInsert_Click(object sender, EventArgs e)
{
    DialogResult result = MessageBox.Show("Do you want to Confirm?", "Confirmation",
    MessageBoxButtons.YesNoCancel);
    if (result == DialogResult.Yes)
    {
        taCellProd.Insert(txtCellDescrip.Text, Convert.ToDecimal(txtCellPrice.Text),
        comboCellProvider.GetItemText(this.comboCellProvider.SelectedItem));
        MessageBox.Show("New Cell Phone Added");
        taCellProd.Fill(dsCell.TblProduct); }
    else if (result == DialogResult.No)
    {
        MessageBox.Show("New Cell Phone NOT Added");
    }
    else
    {
        MessageBox.Show("Cancelled");
    }
}
```

...the new phone should be inserted into the database table

Products
Orders

Cell Phones

ProdID	Description	Price	Provider
2	Samsung Galaxy S9	14000.00	Cell C
3	Samsung Note 9	18500.00	Vodacom
4	Apple iPhone 8	7800.00	MTN
5	Apple iPhone 9	9500.00	MTN
6	Apple iPhone 10	15000.00	Vodacom
7	Apple iPhone 10	14500.00	Cell C
10	Huawei Mate 10	12000.00	Cell C
11	Huawei Mate 10	13000.00	Vodacom
12	Huawei Mate 20 Pro	22000.00	Vodacom

Add New Phone

Provide Phone Description

Select Provider

Vodacom

Enter Phone Price

Add Phone to Stock

PART TWO (COMPULSORY)

- On the 2nd Tab, add a text box to enable searching of the Cell phone list. The search must use predictive searching. As each letter is entered, all the matching phone descriptions must be displayed. However, the challenge this time is to **display the phone details together with the warranty provided by the cell phone provider**. In order to do this an inner join is required. ***This is achieved by opening up the Dataset class and dragging and dropping a TableAdapter into the Dataset.*** The wizard guides you through the rest of the processing. A Sql statement is required to generate the inner join. However, this can be done via the querybuilder

TblProduct

- * (All Columns)
- ☒ ProdID
- ☒ Description
- ☒ Price
- ☒ Provider

TblSupplier

- * (All Columns)
- ☐ Provider
- ☒ Warranty

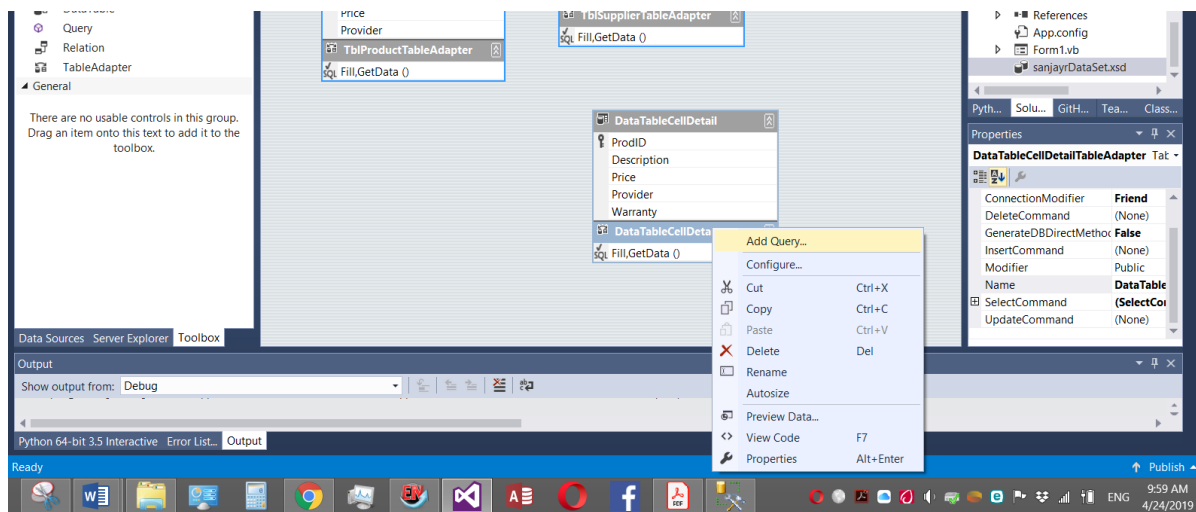
Column	Alias	Table	Output	Sort Type	Sort Order	Filter	Or...
ProdID		TblProduct	☒				
Description		TblProduct	☒				
Price		TblProduct	☒				

```

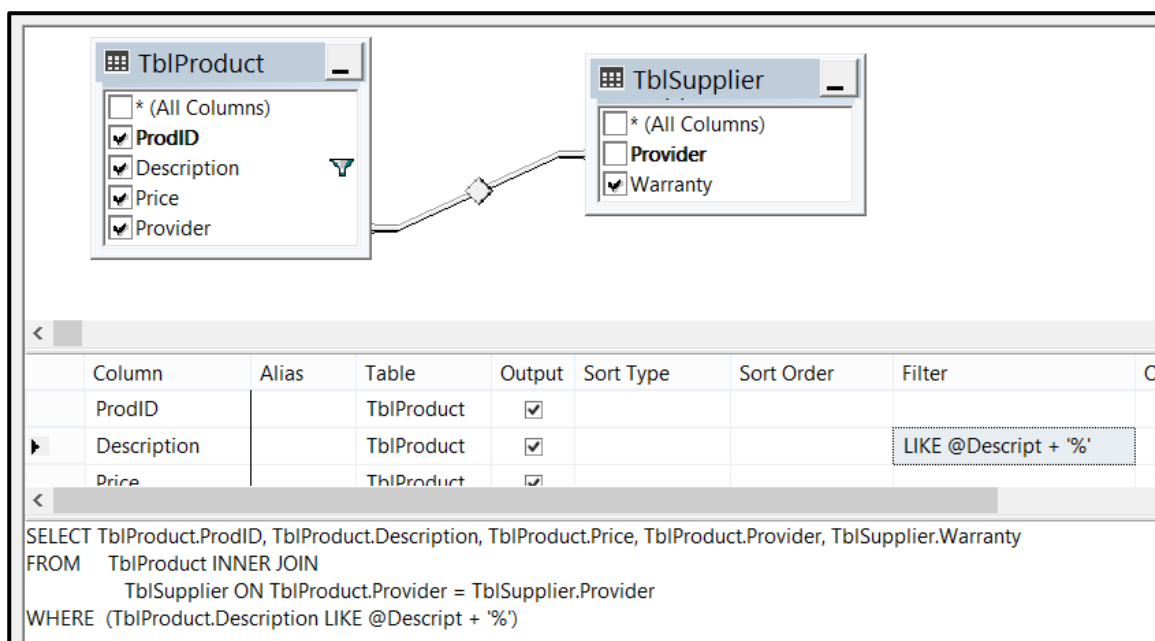
SELECT TblProduct.ProdID, TblProduct.Description, TblProduct.Price, TblProduct.Provider, TblSupplier.Warranty
FROM TblProduct INNER JOIN
      TblSupplier ON TblProduct.Provider = TblSupplier.Provider

```

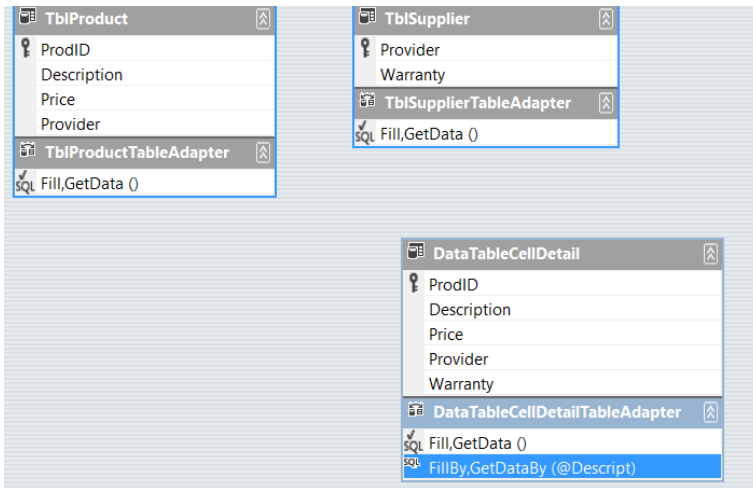
In order to implement the Search Query, add a new method onto the newly added Table Adapter.



Configure the query builder to use a wildcard search and the LIKE Sql operator



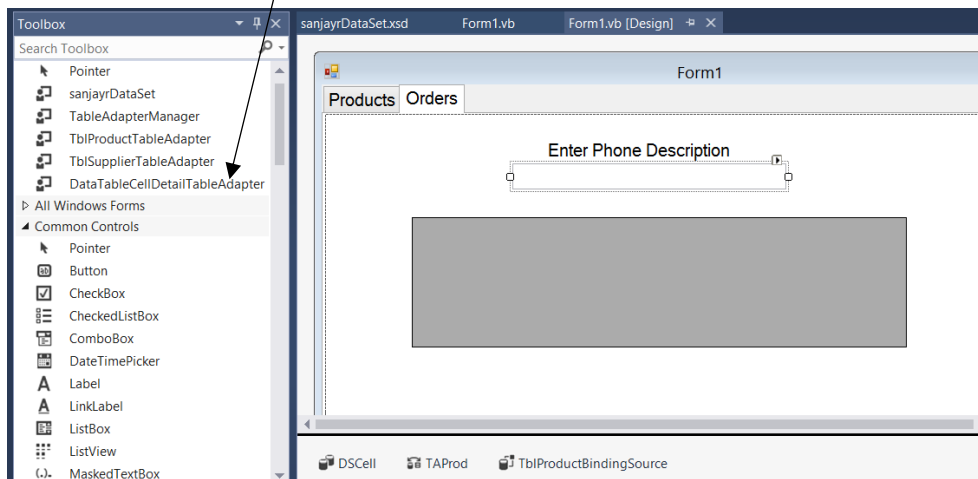
...and the dataset should now look like...



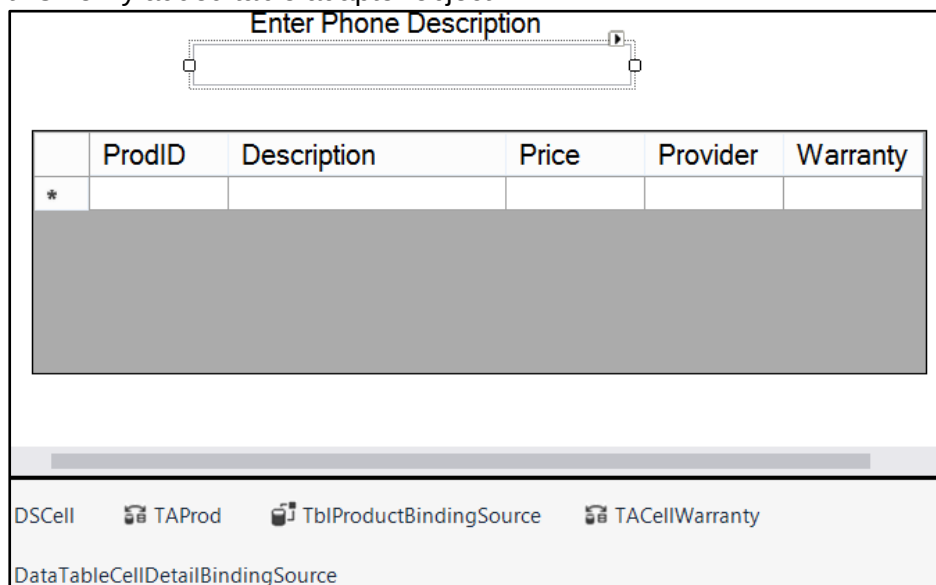
Now **Rebuild** the application so that the newly added Table Adapter becomes available as an object

should be available in the Toolbox

The newly added table adapter



Drag and drop this TableAdapter onto the tray of your application and link up the Gridview to this newly added table adapter object



6. You need to program the event of the user entering a character in the textbox. This is achieved by using the **textchanged event handler** and the search method that was introduced in Prac 4 and Prac 5

```
private void txtSearchCell_TextChanged(object sender, EventArgs e)
{
    taCellDetail2.FillBy(dsCell.DataTableCellDetail, txtSearchCell.Text);
}
```

...to obtain the following effect...

Enter Phone Description

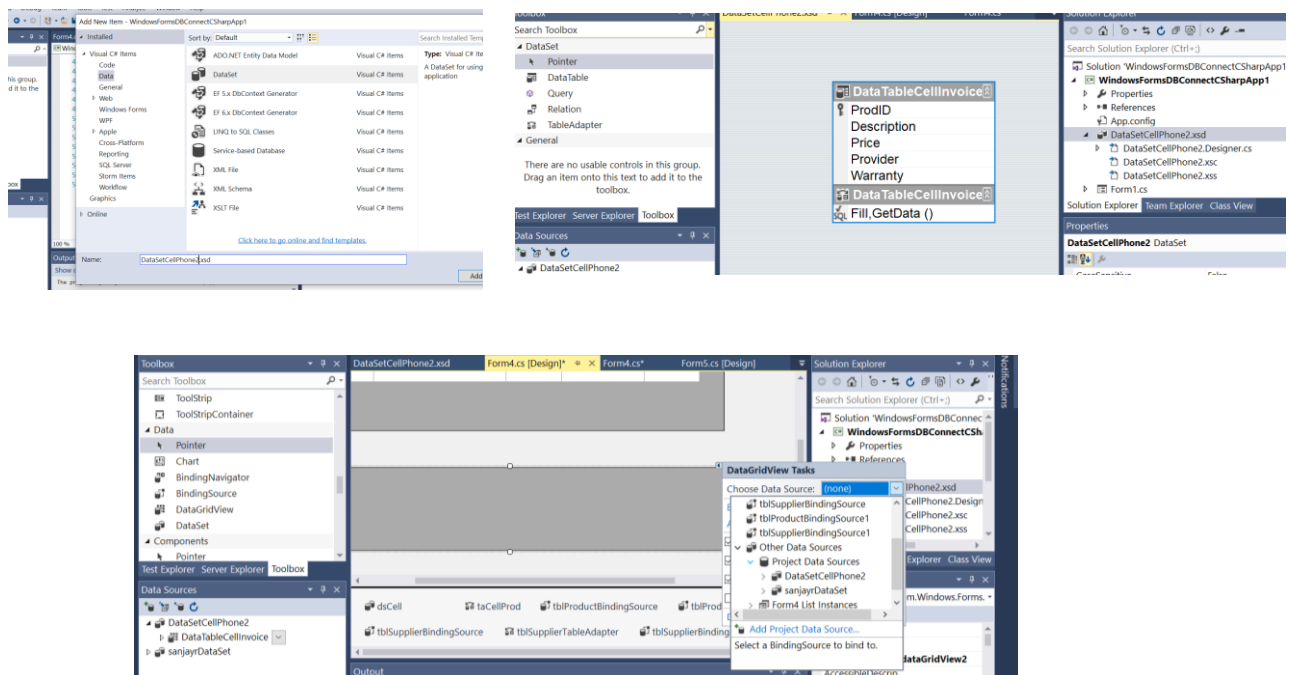
a

	ProdID	Description	Price	Provider	Warranty
▶	4	Apple iPhone 8	7800.00	MTN	24 months
	5	Apple iPhone 9	9500.00	MTN	24 months
	6	Apple iPhone 10	15000.00	Vodacom	36 months
	7	Apple iPhone 10	14500.00	Cell C	18 months
*					

This is now the completion of the PART TWO of this prac*

PART THREE (SEMI-COMPULSORY)

7. The final part is the generation of orders. To implement this functionality, you will need to add a **new Gridview and then add a new dataset object. Also, add a new table adapter object that is identical to the one used in Question 5.** The reason for adding these new objects is that the Order Gridview needs to be separate from the Search Gridview. The sequence of interactions is that as each **row header** from the Search Gridview is double clicked, that row is added to the Orders Gridview.



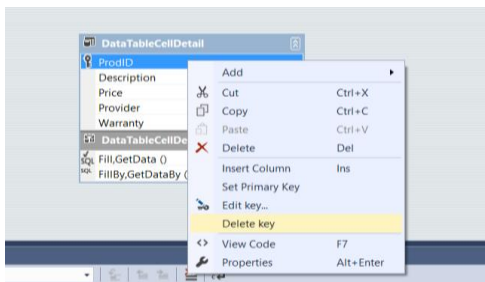
8. The interface should be crafted to have 2 gridviews that are attached to 2 separate datasets...and now looks like...

ProdID	Description	Price	Provider	Warranty
4	Apple iPhone 8	7800.00	MTN	24 months
5	Apple iPhone 9	9500.00	MTN	24 months
6	Apple iPhone 10	15000.00	Vodacom	36 months
7	Apple iPhone 10	14500.00	Cell C	18 months

ProdID	Description	Price	Provider	Warranty

Clear Invoice Amount Owing

9. The processing of the order entails:
- Double clicking the relevant row header
 - A new entry/row should be added to the Order Gridview
 - The Amount Owing should be calculated and updated
10. Find the *Double click of the row header* event handler and copy the entire row from the Search Gridview onto the Orders Gridview. This is achieved by leveraging the DataRow data type.
- ...just a word of caution, if the same product is selected twice, an error will be generated because of a Primary Key violation. This error can be removed by going to the dataset and removing the Primary Key constraint.



```
private void dgvCellProvider_RowHeaderMouseDoubleClick(object sender,
DataGridViewCellEventArgs e)
{
    DataRow dr;
    dr = dsCellPhone2.DataTableCellInvoice.NewRow();
    for (int i = 0; i < dr.ItemArray.Length; i++)
    {
        dr[i] = dgvCellProvider.CurrentRow.Cells[i].Value;
    }
    dsCellPhone2.DataTableCellInvoice.Rows.Add(dr);
}
```

Note: In order to add rows onto a Gridview, you actually add it to the underlying dataset

11. The final part is to write a function/method that will **add the total amount** owing for each item in the order list. As soon as a new row is added, this total should be updated. To achieve this, a function can be used.

```
private double getTotal()
{
    double sum=0;
    foreach (DataRow row in dsCellPhone2.DataTableCellInvoice.Rows)
    {
        sum += Convert.ToDouble(row.ItemArray[2].ToString());
    }
    return sum;
}
```

12. Now Add this method to the end of the code for the *RowHeaderMouseDoubleClick* event handler

```
private void dgvCellProvider_RowHeaderMouseDoubleClick(object sender, DataGridViewCellMouseEventArgs e)
{
    DataRow dr;
    dr = dsCellPhone2.DataTableCellInvoice.NewRow();
    for (int i = 0; i < dr.ItemArray.Length; i++)
    {
        dr[i] = dgvCellProvider.CurrentRow.Cells[i].Value;
    }
    dsCellPhone2.DataTableCellInvoice.Rows.Add(dr);
    txtAmtOwing.Text = getTotal().ToString("C2");
}
```

13. To clear the Order rows ...clear the rows from the Order Dataset....DSOrder.DataTableCellDetail.Rows.clear()..and clear the total in the textbox by txtTotal.clear()

```
private void btnClearInvoice_Click(object sender, EventArgs e)
{
    dsCellPhone2.Clear();
}
```

PART FOUR (SEMI-COMPULSORY)

Now to Record the Order...

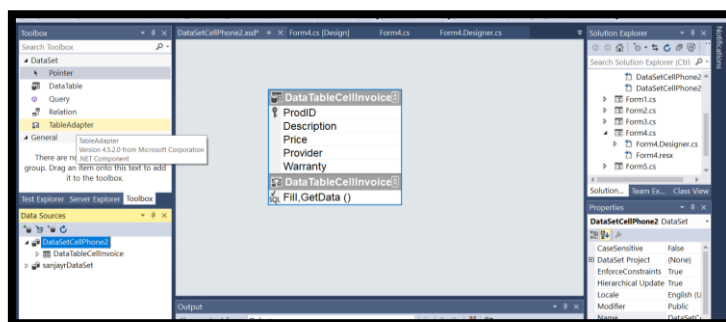
The First Part will adopt a rather simplistic approach...this is where we record the overall sale...i.e. just the total amount for the sale...in the next version we will record the sale detail as well

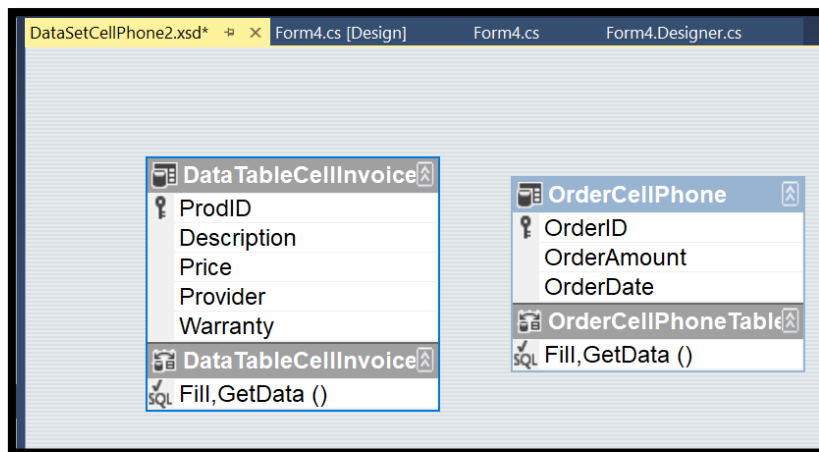
14. First step is to create a database table for the sale

Column Name	Data Type	Allow Nulls
OrderID	int	☐
OrderAmount	decimal(10, 2)	☒
OrderDate	date	☒
		☐

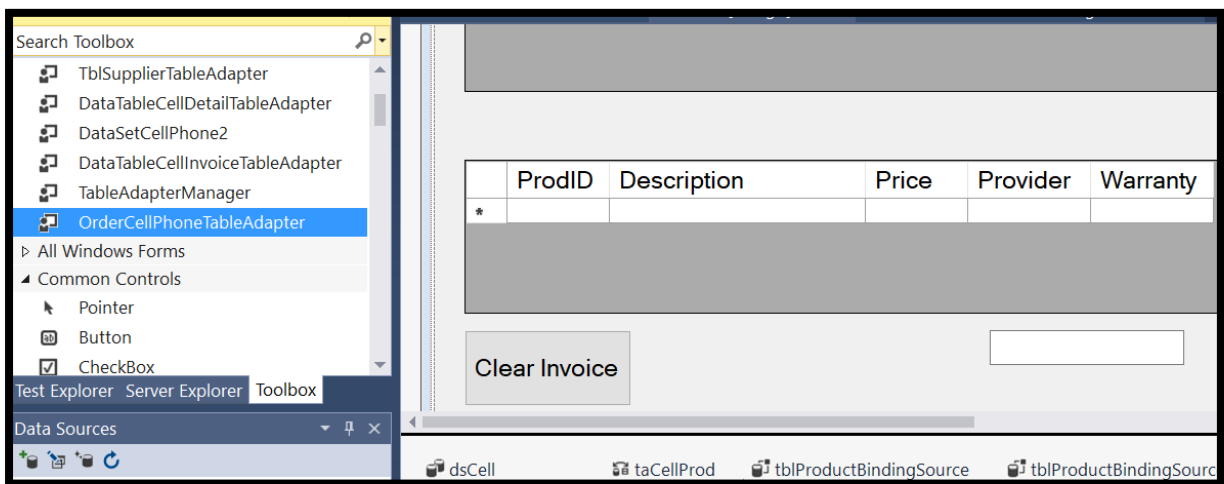
Column Properties	
DTS-published	No
> Full-text Specification	No
Has Non-SQL Server Subscriber	No
> Identity Specification	Yes
Indexable	Yes

15. Now you need to establish a datasource to this newly added table
This can be done using the TableAdapter object and adding it to an already existing dataset...preferable to use the 2nd





Rebuild the Application...so that you see the OrderCellPhone Table Adapter in the Toolbox



Drag the OrderCellPhoneTable Adapter onto the tray (bottom) of the form

16. The Recording of the Sale is simply an Insert Operation that is available as standard method of the OrderCellPhone Table Adapter. Invoke the Insert Method of the OrderCellPhone Table Adapter...and thenclear the Invoice (because the Order would now be recorded and you want to make the invoice available again for the recording of a new Order)...by clearing the rows from the Dataset

Inserting the New Record

In C# this is a little bit tricky because of the stringent observation of Data Types. Usually an Insert would simply entail an invocation of the Insert Method and the passing in of all the data parameters required for the Insert. Firstly, on the Form, the amount owing is stored with a currency symbol. The currency symbol must be removed when you use the Insert methods of the Table Adapter. To do this you have to add a reference to the namespace called *globalisation*

```
using System.Globalization;
```

and the code will like...

```

private void btnRecordCellPhoneSale_Click(object sender, EventArgs e)
{
    DialogResult result = MessageBox.Show("Do you want to Confirm?", "Confirmation",
    MessageBoxButtons.YesNoCancel);
    if (result==DialogResult.Yes)
    {
        taOrderCellPhone.Insert(Decimal.Parse(txtAmtOwing.Text, NumberStyles.Currency),
        DateTime.Now.Date); //Inserts the Order into the OrderCellPhone table
        taOrderCellPhone.Fill(dsCellPhone2.OrderCellPhone); //Refreshes the Gridview to
        display the newly inserted record
    }
}

```

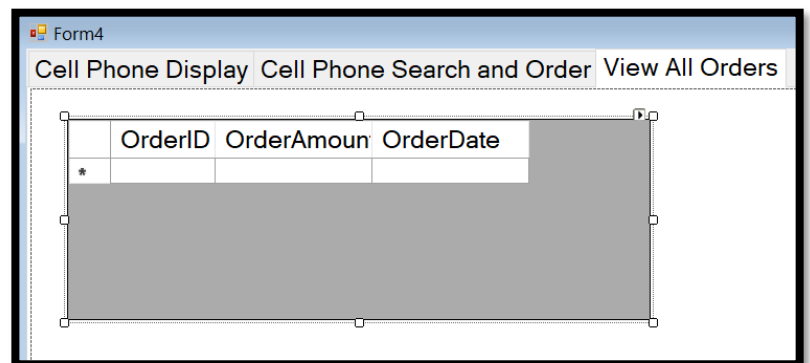
17. Finally add a new tab that displays all the recorded sales events. To display all recorded sales, you need to add a Gridview and use the OrderCellPhone Table Adapter to fill the OrderData table with the Orders that have been recorded.

```

private void Form4_Load(object sender, EventArgs e)
{
    taOrderCellPhone.Fill(dsCellPhone2.OrderCellPhone);
    this.tblSupplierTableAdapter.Fill(this.dsCell.TblSupplier);
    taCellProd.Fill(dsCell.TblProduct);
}

```

On the newly added tab...set up a gridview as shown to show ALL the Orders....



The Sequence of processing will now entail....

18. Search for a cell phone; Double click to select the sale item; The items get added to the invoice

h

ProdID	Description	Price	Provider
10	Huawei Mate 10	R12,000....	Cell C
11	Huawei Mate 10	R13,000....	Vodacorr
12	Huawei Mate 20 Pro	R22,000....	Vodacorr

ProdID	Description	Price	Provide
5	Apple iPhone 9	R9,500.00	MTN
11	Huawei Mate 10	R13,000....	Vodacorr

19. Record the Cell Phone Sale....Invoice gets cleared and View the newly recorded sale

Cell Phone Display Cell Phone Search and Order View All Orders

OrderID	OrderAmount	OrderDate
10	R13,000.00	17-May-21
11	R39,000.00	17-May-21
12	R22,500.00	17-May-21

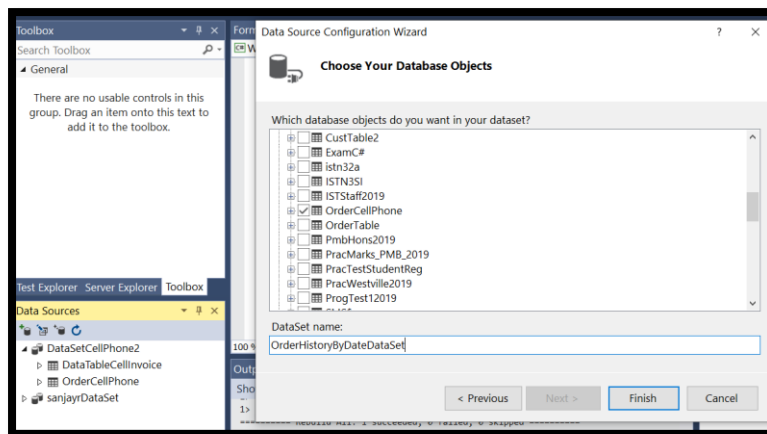
PART FIVE (OPTIONAL)

20. Here a new tab will be added to allow users to view sales according to a date that is selected off the DateTime picker control; As soon as a date is selected the cell phone sales for that day only is made available for viewing

To enable this bit of processing, you need to establish a New Data Source connection and create a link to the Order table in the database. As soon as this happens, rebuild your application and then the newly established data source is made visible together with the dataset and table adapter for the Order table...you are now ready to go!!

Here are a few screenshots that will assist!

Creating the New Data Source

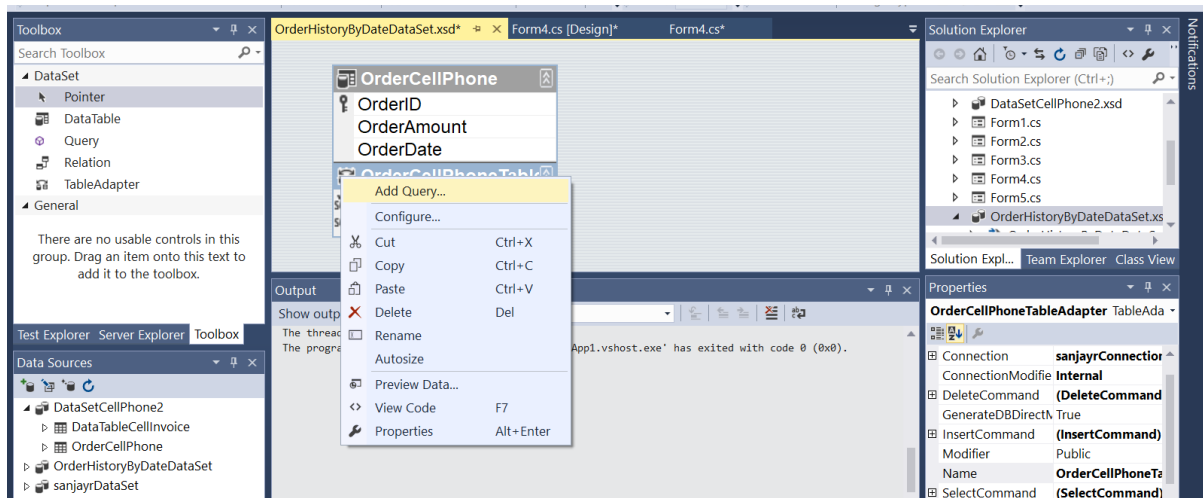


Rebuild the Application...drag and drop the dataset and tableadapter objects into the tray of the form

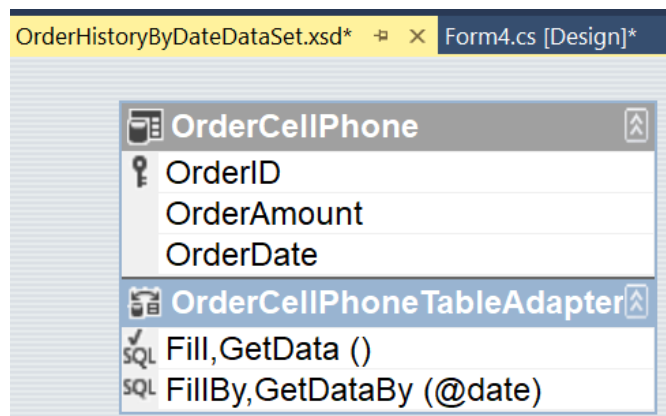
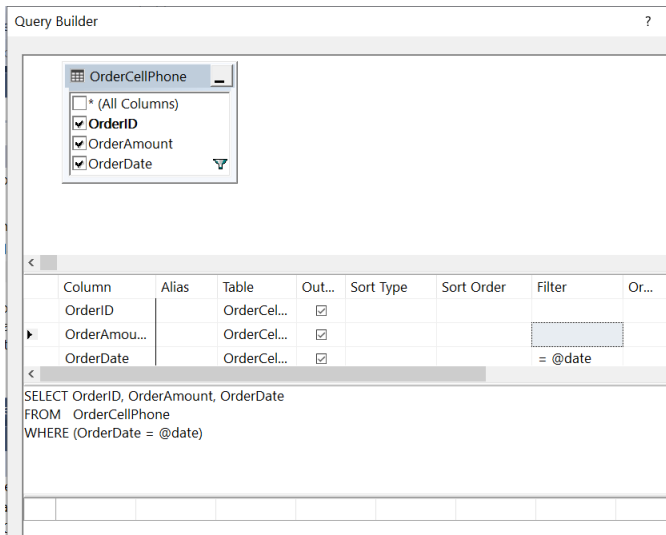
Fill the dataset with data from the Order table....this will be done inside the Form_Load event handler

```
private void Form4_Load(object sender, EventArgs e)
{
    taOrderCellPhoneHistByDate.Fill(dsOrderHistByDate.OrderCellPhone);
}
```

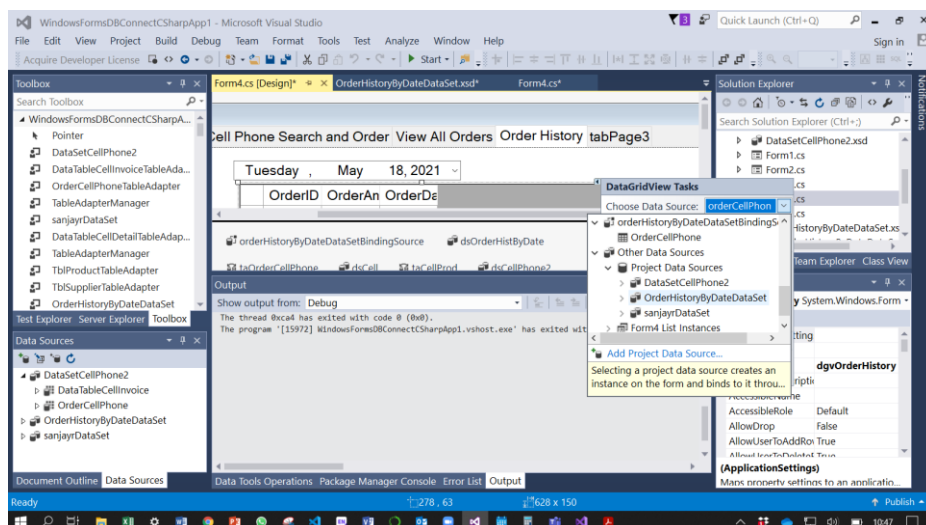
The default Fill method fills ALL the Order data...we want this customised by date...so add another query on the dataset to achieve this filtered view of the Order data



Adding a parameterised Query



...and so you have an extra method on the table adapter object
Now arrange the interface....add a datetime picker control and a gridview...set the datasource for the gridview to be the Project>>>Dataset>>>Order table



By default this Gridview will be loaded with all the orders; however if you position a date time picker control on the form and select a date, then the gridview data should be filtered according to that date
The Interface now looks like....

Cell Phone Display Cell Phone Search and Order View All Orders Order History

Tuesday , May 18, 2021 ▾

	OrderID	OrderAmount	OrderDate
*			

When a date is selected (on the datetimepicker – dtp1) only the Order for that day is displayed

```
private void dtp1_ValueChanged(object sender, EventArgs e)
{
    taOrderCellPhoneHistByDate.FillBy(dsOrderHistByDate.OrderCellPhone,
    dtp1.Value.Date.ToShortDateString());
}
```

Tuesday , May 18, 2021 ▾

	OrderID	OrderAmount	OrderDate
▶	13	28000.00	18-May-21
*			

Monday , May 17, 2021 ▾

	OrderID	OrderAmount	OrderDate
▶	6	R5,000.00	17-May-21
	7	R7,500.00	17-May-21
	8	R9,500.00	17-May-21
	9	R9,500.00	17-May-21

And finally, if you wanted to be a bit more fancy, you could write code so that as soon as the **OrderHistory Tab is entered**, the orders for the current day only is displayed!!

Tuesday , May 18, 2021			
	OrderID	OrderAmount	OrderDate
	13	R28,000.00	18-May-21
▶	14	R20,800.00	18-May-21
*			